

Dynamic Pricing Using RL

Week 5 — Deep RL: DQN & PPO + Self-Play

This week, we move from tabular Q-learning (Weeks 3-4) to deep RL, since the price-state space becomes too large to tabulate as complexity grows. Implemented DQN and PPO using Stable-Baselines3, and set up simultaneous self-play between two learning agents in the custom pricing Gymnasium environment.

Key concepts learned

- **DQN**: uses a neural network to approximate $Q(s,a)$, stabilized by two tricks:
 - **experience replay** breaks correlation between consecutive samples by sampling randomly from a replay buffer
 - **target network** a slowly-updated copy of the Q-network used to compute targets, preventing a moving-target feedback loop that destabilizes training.
- **PPO**: an actor-critic policy gradient method that constrains policy updates using a clipped surrogate objective, preventing the new policy from moving too far from the old one in a single update. This avoids catastrophic policy collapse that plain policy gradient methods suffer from.
- **Self-play non-stationarity**: when both agents update simultaneously, each agent's "environment" (the opponent) keeps changing under it, breaking the stationarity assumption most RL convergence guarantees rely on. Mitigated this by freezing one agent every N steps while the other trains (following Heinrich & Silver, 2016).

What the code does

1. `PricingDuopolyEnv`: a two-firm repeated Bertrand-style pricing environment with logit demand (same demand structure as Calvano et al., 2020).
2. Wraps the environment in SB3's `DummyVecEnv` (required by SB3 or it throw a cryptic shape-mismatch error).
3. Trains two agents in alternating rounds: Agent A trains for 5000 steps against a frozen Agent B, then Agent B trains for 5000 steps against the now-frozen Agent A, repeating until 100,000 total steps.
4. Logs price trajectories to check for convergence, loss going down does NOT mean prices have converged, so the price trajectory plot is the real signal to watch (a flat loss with chaotic prices means the agent is stuck).